

Sécurité API

Auteur : Jonatan NSUALU

Date : Mai 2026

[Introduction](#)

[Authentification JWT](#)

[Autorisations et rôles](#)

[Sécurisation des mots de passe](#)

[Protection des routes API](#)

[Validation des données](#)

[Configuration CORS](#)

[Gestion des erreurs](#)

[Variables d'environnement et configuration sécurisée](#)

Introduction

La sécurisation de l'API est essentielle afin de protéger les données de l'entreprise ainsi que les informations des utilisateurs.

Une API non sécurisée peut exposer le système à des attaques, des accès non autorisés ou des fuites de données sensibles.

Dans ConnecTeam, plusieurs mécanismes de sécurité seront mis en place afin de :

- contrôler les accès ;
 - protéger les données utilisateurs ;
 - sécuriser les échanges ;
 - limiter les risques d'attaques ;
 - garantir la confidentialité des informations.
-

Authentification JWT

L'API utilise JWT (*JSON Web Token*) pour authentifier les utilisateurs.

JWT permet de générer un token sécurisé contenant les informations nécessaires à l'authentification de l'utilisateur.

Après la connexion, ce token est envoyé au client puis transmis à chaque requête protégée.

Cette solution a été choisie car elle permet :

- une authentification stateless ;
- une meilleure scalabilité ;
- une intégration simple avec les API REST ;
- une communication sécurisée entre le frontend Angular et le backend Spring Boot.

Contrairement aux sessions classiques stockées côté serveur, JWT permet au serveur de ne pas conserver l'état de connexion de chaque utilisateur, ce qui améliore les performances et l'évolutivité de l'application.

Autorisations et rôles

Le système utilise une gestion des permissions basée sur les rôles afin de limiter les accès selon les responsabilités des utilisateurs.

Les rôles principaux sont :

- `ROLE_ADMIN`
- `ROLE_MANAGER`
- `ROLE_EMPLOYEE`
- `ROLE_HR`

Chaque rôle possède des permissions spécifiques définies dans l'application.

Exemples :

- un employé ne peut pas accéder aux fonctionnalités d'administration ;
- un manager peut gérer les tâches de son équipe ;
- les RH peuvent gérer les utilisateurs et les actualités ;

- l'administrateur possède un accès complet au système.
-

Sécurisation des mots de passe

Les mots de passe des utilisateurs ne sont jamais stockés en clair dans la base de données.

Ils sont sécurisés grâce à un système de hash utilisant l'algorithme BCrypt.

Le hash permet de transformer un mot de passe en une valeur chiffrée irréversible afin de protéger les informations sensibles des utilisateurs en cas de fuite de données.

BCrypt a été choisi car :

- il est reconnu comme sécurisé ;
- il résiste aux attaques par force brute ;
- il ajoute automatiquement un salt de sécurité ;
- il est largement utilisé dans les applications professionnelles modernes.

L'application imposera également des mots de passe forts afin d'améliorer la sécurité des comptes utilisateurs.

Les mots de passe devront contenir :

- un minimum de caractères ;
- des lettres majuscules et minuscules ;
- des chiffres ;
- des caractères spéciaux.

Cette politique permet de réduire les risques :

- d'attaques par dictionnaire ;
 - de mots de passe trop faibles ;
 - de compromission des comptes utilisateurs.
-

Protection des routes API

Les routes de l'API doivent être protégées afin d'empêcher les accès non autorisés aux ressources sensibles.

Certaines routes seront accessibles uniquement aux utilisateurs authentifiés ou possédant un rôle spécifique.

Exemples :

- endpoints administrateur sécurisés ;
- accès restreint selon les rôles ;
- contrôle des permissions sur les ressources.

Cette protection permet de limiter :

- les accès illégitimes ;
- les tentatives d'usurpation ;
- les modifications non autorisées ;
- les risques d'exposition des données sensibles.

La sécurisation des routes sera mise en place avec Spring Security via :

- l'authentification JWT ;
 - des filtres de sécurité ;
 - des règles d'autorisation selon les rôles ;
 - des annotations de sécurité sur certaines routes.
-

Validation des données

Les données envoyées à l'API seront validées avant traitement afin de garantir leur cohérence et leur sécurité.

Pour cela, l'application utilisera des DTO (*Data Transfer Object*).

Les DTO permettent :

- de contrôler les données reçues ;
- de séparer les données exposées des entités internes ;
- de limiter les informations envoyées au client ;

- de renforcer la sécurité de l'API.

Des contraintes de validation seront également appliquées :

- email valide ;
- taille minimale des mots de passe ;
- champs obligatoires ;
- limites de taille sur certains contenus.

Cette validation permet de se protéger contre :

- les données invalides ;
 - les erreurs de saisie ;
 - certaines attaques malveillantes ;
 - l'injection de données non conformes.
-

Configuration CORS

Le CORS (*Cross-Origin Resource Sharing*) est un mécanisme de sécurité permettant d'autoriser ou de bloquer les requêtes provenant d'un autre domaine.

Dans le cadre de ConnecTeam, le frontend Angular et le backend Spring Boot fonctionneront sur des adresses différentes durant le développement.

Exemple :

- Frontend → `localhost:4200`
- Backend → `localhost:8080`

Sans configuration CORS, le navigateur bloquerait les requêtes entre les deux applications.

Le CORS sera principalement configuré côté backend avec Spring Security afin d'autoriser uniquement les domaines approuvés.

Côté frontend Angular, les routes de navigation seront gérées via le système de routing Angular permettant :

- la navigation entre les pages ;
- la protection de certaines routes ;

- la restriction d'accès selon l'authentification.

Cette protection sera mise en place grâce aux `Route Guards` d'Angular.

Gestion des erreurs

La gestion des erreurs permet de contrôler les problèmes rencontrés par l'application et de retourner des réponses compréhensibles au frontend.

Une bonne gestion des erreurs permet :

- d'éviter les plantages ;
- d'améliorer l'expérience utilisateur ;
- de faciliter le débogage ;
- de sécuriser les informations sensibles.

Les réponses API utiliseront des réponses HTTP standardisées avec :

- un code HTTP ;
- un message d'erreur ;
- une structure de réponse commune (`ApiResponse`).

Exemples :

- `400 Bad Request`
- `401 Unauthorized`
- `403 Forbidden`
- `404 Not Found`
- `500 Internal Server Error`

Cette approche permet au frontend de gérer correctement les différents cas d'erreurs et d'afficher des messages adaptés aux utilisateurs.

Variables d'environnement et configuration sécurisée

Afin d'éviter d'exposer des informations sensibles dans le code source, certaines données de configuration seront externalisées dans des fichiers de

configuration ou des variables d'environnement.

Cela concerne notamment :

- les clés secrètes JWT ;
- les identifiants de connexion à la base de données ;
- les mots de passe de services ;
- certaines configurations sensibles de l'application.

Grâce à la gestion de configuration de IntelliJ IDEA et de Spring Boot, ces informations pourront être stockées dans des fichiers séparés ou injectées via des variables d'environnement au démarrage de l'application.

```
DB_USERNAME=${DB_USERNAME}
DB_PASSWORD=${DB_PASSWORD}
JWT_SECRET=${JWT_SECRET}
```

Cette approche permet :

- d'éviter de stocker des secrets dans le dépôt Git ;
- de renforcer la sécurité du projet ;
- de faciliter le changement de configuration entre développement et production ;
- de protéger les données sensibles en cas de partage du code source.

En environnement de production, les informations sensibles seront configurées directement sur le serveur ou dans les variables d'environnement du système afin d'améliorer la sécurité globale de l'application.